

```
// This work is licensed under a Attribution-NonCommercial-ShareAlike 4.0
International (CC BY-NC-SA 4.0) https://creativecommons.org/licenses/by-nc-sa/4.0/
// © LuxAlgo
```

```
//@version=5
```

```
indicator("Swing Volume Profiles [LuxAlgo]", "LuxAlgo - Swing Volume Profiles", true,
max_bars_back = 5000, max_boxes_count = 500, max_labels_count = 500,
max_lines_count = 500)
```

```
//-----
//Settings
//-----{
```

```
mode = input.string('Present', title = 'Mode', options = ['Present', 'Historical'], inline =
'MOD')
back = input.int (300, ' # Bars', minval = 100, maxval = 5000, step = 10, inline =
'MOD')
```

```
grpVP = 'Swing Volume Profiles'
```

```
ppTT = 'The Swing High Low indicator is used to determine and anticipate potential
changes in market price and reversals\n' +
```

```
    '\Swing Volume Profiles [LuxAlgo]' aims at highlighting the trading activity at
specified price levels between two Swing Levels'
```

```
ppLen = input.int(47, "Swing Detection Length", minval = 1, group = grpVP, tooltip =
ppTT)
```

```
vpTT = 'Common Interest Profile (Total Volume) - displays total trading activity over
a specified time period at specific price levels'
```

```
vpShw = input.bool(true, 'Swing Volume Profiles', inline = 'BB3', group = grpVP,
tooltip = vpTT)
```

```
vpTVC = input.color(color.new(#fbc02d, 65), "", inline = 'BB3', group = grpVP)
```

```
vpVVC = input.color(color.new(#434651, 65), "", inline = 'BB3', group = grpVP)
```

```
vpB = input.bool(false, 'Profile Range Background Fill', inline = 'BG', group =
grpVP)
```

```
vpBC = input.color(color.new(#2962ff, 95), "", inline = 'BG', group = grpVP)
```

```
grpPC = 'Point of Control (POC)'
```

```
pcTT = 'Point of Control (POC) - The price level for the time period with the highest
traded volume'
```

```
pcShw = input.bool(true, 'Point of Control (PoC)', inline = 'PoC', group = grpPC,
tooltip = pcTT)
```

```
pcC = input.color(color.new(#ff0000, 0), "", inline = 'PoC', group = grpPC)
```

```
dpTT = 'Developing Point of Control, displays how POC is changing during the
active market session'
```

```
dpcS = input.bool(true, 'Developing PoC ', inline = 'dPoC', group = grpPC, tooltip
= dpTT)
```

```
dpcC = input.color(color.new(#ff0000, 0), "", inline = 'dPoC', group = grpPC)
```

```
pcE = input.string('None', 'Extend PoC', options=['Until Last Bar', 'Until Bar Cross', 'Until Bar Touch', 'None'], group = grpPC)
```

```
grpVA = 'Value Area (VA)'
```

```
vaTT = 'Value Area (VA) – The range of price levels in which a specified percentage of all volume was traded during the time period'
```

```
isVA = input.float(68, "Value Area Volume %", minval = 0, maxval = 100, group = grpVA, tooltip = vaTT) / 100
```

```
vhTT = 'Value Area High (VAH) - The highest price level within the value area'
```

```
vhShw = input.bool(true, 'Value Area High (VAH)', inline = 'VAH', group = grpVA, tooltip = vhTT)
```

```
vaHC = input.color(color.new(#2962ff, 0), "", inline = 'VAH', group = grpVA)
```

```
viTT = 'Value Area Low (VAL) - The lowest price level within the value area'
```

```
viShw = input.bool(true, 'Value Area Low (VAL)', inline = 'VAL', group = grpVA, tooltip = viTT)
```

```
vaLC = input.color(color.new(#2962ff, 0), "", inline = 'VAL', group = grpVA)
```

```
vaB = input.bool(false, 'Value Area (VA) Background Fill', inline = 'vBG', group = grpVA)
```

```
vaBC = input.color(color.new(#2962ff, 89), "", inline = 'vBG', group = grpVA)
```

```
grpLQ = 'Liquidity Levels / Voids'
```

```
liqUF = input(true, 'Unfilled Liquidity, Thresh', inline = 'UFL', group = grpLQ)
```

```
liqT = input(21, "", inline = 'UFL', group = grpLQ) / 100
```

```
liqC = input.color(color.new(#00bcd4, 90), "", inline = 'UFL', group = grpLQ)
```

```
grpLB = 'Profile Stats'
```

```
ppLev = input.string('Swing High/Low', 'Position', options = ['Swing High/Low', 'Profile High/Low', 'Value Area High/Low'], inline='ppLS', group = grpLB)
```

```
ppLS = input.string('Small', "Size", options=['Tiny', 'Small', 'Normal'], inline='ppLS', group = grpLB)
```

```
ppS = switch ppLS
```

```
    'Tiny' => size.tiny
```

```
    'Small' => size.small
```

```
    'Normal' => size.normal
```

```
ppP = input(false, "Price", inline = 'Levels', group = grpLB)
```

```
ppC = input(false, "Price Change", inline = 'Levels', group = grpLB)
```

```
ppV = input(false, "Cumulative Volume", inline = 'Levels', group = grpLB)
```

```
grpOT = 'Volume Profile Others'
```

```
vpLev = input.int(27, 'Number of Rows', minval = 10, maxval = 100, step = 1, group = grpOT)
```

```
vpPlc = input.string('Left', 'Placment', options = ['Right', 'Left'], group = grpOT)
```

```
vpWth = input.int(50, 'Profile Width %', minval = 0, maxval = 100, group = grpOT) / 100
```

```

//-----}
//User Defined Types
//-----{

// @type      bar properties with their values
//
// @field h    (float) high price of the bar
// @field l    (float) low price of the bar
// @field v    (float) volume of the bar
// @field i    (int)  index of the bar

type bar
    float h = high
    float l = low
    float v = volume
    int  i = bar_index

// @type      store pivot high/low and index data
//
// @field x    (int)  last pivot bar index
// @field x1   (int)  previous pivot bar index
// @field h    (float) last pivot high
// @field l    (float) last pivot low
// @field s    (string) last pivot as 'L' or 'H'

type pivotPoint
    int  x
    int  x1
    float h
    float l
    string s

// @type      maintain liquidity data
//
// @field b    (array<bool>) array maintains price levels where liquidity exists
// @field bx   (array<box>) array maintains visual object of price levels where
liquidity exists

type liquidity
    bool [] b
    box  [] bx

// @type      maintain volume profile data
//
// @field vs   (array<float>) array maintains total traded volume
// @field vp   (array<box>) array maintains visual object of each price level

type volumeProfile
    float [] vs

```

```

    box [] vp

//-----}
//Variables
//-----{

var aPOC = array.new_box()
var dPOC = array.new_line()
var dPCa = array.new_line()

var laP = 0, var lbP = 0, var dpcl = 0, var float ersten = na

bar b = bar.new()

var pivotPoint pp = pivotPoint.new()

var liquidity[] aLIQ = array.new<liquidity> (1, liquidity.new(array.new <bool> (vpLev,
false), array.new <box> (na)))
var liquidity[] dLIQ = array.new<liquidity> (1, liquidity.new(array.new <bool>
(na) , array.new <box> (na)))

volumeProfile aVP = volumeProfile.new(array.new <float> (vpLev + 1, 0.),
array.new <box> (na))
volumeProfile aVPa = volumeProfile.new(array.new <float> (vpLev + 1, 0.),
array.new <box> (na))
var volumeProfile dVP = volumeProfile.new(array.new <float> (na) , array.new
<box> (na))

//-----}
//Functions/methods
//-----{

// @function    calcuates highest, lowest price value and cumulative volume of the
//              given range
//
// @param _l    (int) length of the range
// @param _c    (bool) check
// @param _o    (int) offset
//
// @returns     (float, float, float) highest, lowest and cumulative volume

f_calcHL(_l, _c, _o) =>
    if _c
        l = low [_o]
        h = high[_o]
        v = 0.

        for x = 0 to _l - 1
            l := math.min(low [_o + x], l)
            h := math.max(high[_o + x], h)

```

```

    v += volume[_o + x]

    l := math.min(low [_o + _l], l)
    h := math.max(high[_o + _l], h)

    [h, l, v]

// @function      check bar breaches
//
// @param _a      (array<box>) array containing the boxes to be checked
// @param _e      (strings) extend statment : 'Until Last Bar', 'Until Bar Cross',
'Until Bar Touch' and 'None'
//
// @returns       none, updated visual objects (boxes)

f_checkBreaches(_a, _e) =>
    int qBX = array.size(_a)
    for no = 0 to (qBX > 0 ? qBX - 1 : na)
        if no < array.size(_a)
            cBX = array.get(_a, no)
            mBX = math.avg(box.get_bottom(cBX), box.get_top(cBX))
            ced = math.sign(close[1] - mBX) != math.sign(close - mBX)
            ted = math.sign(close[1] - mBX) != math.sign(low - mBX) or
math.sign(close[1] - mBX) != math.sign(high - mBX)

            if ced and _e == 'Until Bar Cross'
                array.remove(_a, no)
                int(na)
            else if ted and _e == 'Until Bar Touch'
                array.remove(_a, no)
                int(na)
            else
                box.set_right(cBX, bar_index)
                int(na)

// @function      creates new line object, updates existing line objects
//
// @param         details in Pine Script™ language reference manual
//
// @returns       id of the line

f_drawLineX(_x1, _y1, _x2, _y2, _xloc, _extend, _color, _style, _width) =>
    var id = line.new(_x1, _y1, _x2, _y2, _xloc, _extend, _color, _style, _width)
    line.set_xy1(id, _x1, _y1)
    line.set_xy2(id, _x2, _y2)
    line.set_color(id, _color)
    id

// @function      creates new label object, updates existing label objects
//

```

```

// @param      details in Pine Script™ language reference manual
//
// @returns     none

f_drawLabelX(_x, _y, _text, _xloc, _yloc, _color, _style, _textcolor, _size, _textalign,
_tooltip) =>
    var id = label.new(_x, _y, _text, _xloc, _yloc, _color, _style, _textcolor, _size,
_textalign, _tooltip)
    label.set_xy(id, _x, _y)
    label.set_text(id, _text)
    label.set_tooltip(id, _tooltip)

//-----}
//Calculations
//-----{

per  = mode == 'Present' ? last_bar_index - b.i <= back : true
nzV  = nz(b.v)

pp_h = ta.pivohigh(ppLen, ppLen)
pp_l = ta.pivotlow (ppLen, ppLen)

if not na(pp_h)
    pp.h := pp_h
    pp.s := 'H'

if not na(pp_l)
    pp.l := pp_l
    pp.s := 'L'

go = not na(pp_h) or not na(pp_l)

if go
    pp.x1 := pp.x
    pp.x  := b.i

vpLen = pp.x - pp.x1

[pHst, pLst, tV] = f_calcHL(vpLen, go, ppLen)
pStp = (pHst - pLst) / vpLev

[pHta, pLta, _] = f_calcHL(ppLen, go, 0)
pSpa = (pHta - pLta) / vpLev

if go and nzV and pStp > 0 and b.i > vpLen and vpLen > 0 and per
    if dPCa.size() > 0
        for i = 0 to dPCa.size() - 1
            dPCa.shift().delete()

```

```

for blt = 1 to vpLen
  l = 0
  bl = blt + ppLen

  for pLev = pLst to pHst by pStp
    if b.h[bl] >= pLev and b.l[bl] < pLev + pStp
      aVP.vs.set(l, aVP.vs.get(l) + nzV[bl] * ((b.h[bl] - b.l[bl]) == 0 ? 1 : pStp /
(b.h[bl] - b.l[bl])))
      l += 1

  pcL = aVP.vs.indexof(aVP.vs.max())
  ttV = aVP.vs.sum() * isVA
  va = aVP.vs.get(pcL)
  laP := pcL
  lbP := pcL

  while va < ttV
    if lbP == 0 and laP == vpLev - 1
      break

    vaP = 0.
    if laP < vpLev - 1
      vaP := aVP.vs.get(laP + 1)

    vbP = 0.
    if lbP > 0
      vbP := aVP.vs.get(lbP - 1)

    if vbP == 0 and vaP == 0
      break

    if vaP >= vbP
      va += vaP
      laP += 1
    else
      va += vbP
      lbP -= 1

  aLIQ.unshift(liquidity.new(array.new <bool> (vpLev, false), array.new <box> (na)))
  cLIQ = aLIQ.get(0)

  for l = vpLev - 1 to 0
    if vpShw
      sbl = vpPlc == 'Right' ? b.i - int(aVP.vs.get(l) / aVP.vs.max() * vpLen *
vpWth) : b.i - vpLen
      ebl = vpPlc == 'Right' ? b.i : sbl + int( aVP.vs.get(l) / aVP.vs.max() * vpLen *
vpWth)
      aVP.vp.push(box.new(sbl - ppLen, pLst + (l + 0.1) * pStp, ebl - ppLen, pLst +
(l + 0.9) * pStp, l >= lbP and l <= laP ? vpTVC : vpVVC, bgcolor = l >= lbP and l <=
laP ? vpTVC : vpVVC))

```

```

if liqUF
    if aVP.vs.get(l) / aVP.vs.max() < liqT
        cLIQ.b.set(l, true)
        cLIQ.bx.unshift(box.new(b.i[ppLen], pLst + (l + 0.00) * pStp, b.i[ppLen],
pLst + (l + 1.00) * pStp, border_color = color(na), bgcolor = liqC ))
    else
        cLIQ.bx.unshift(box.new(na, na, na, na))
        cLIQ.b.set(l, false)

for blt = 0 to vpLen
    bl = blt + ppLen
    int qBX = cLIQ.bx.size()

    for no = 0 to (qBX > 0 ? qBX - 1 : na)
        if no < cLIQ.bx.size()
            if cLIQ.b.get(no)
                cBX = cLIQ.bx.get(no)
                mBX = math.avg(cBX.get_bottom(), cBX.get_top())

                if math.sign(close[bl + 1] - mBX) != math.sign(low[bl] - mBX) or
math.sign(close[bl + 1] - mBX) != math.sign(high[bl] - mBX) or math.sign(close[bl +
1] - mBX) != math.sign(close[bl] - mBX)
                    cBX.set_left(b.i[bl])
                    cLIQ.b.set(no, false)

for bl = ppLen to 0
    int qBX = cLIQ.bx.size()

    for no = (qBX > 0 ? qBX - 1 : na) to 0
        if no < cLIQ.bx.size()
            cBX = cLIQ.bx.get(no)
            mBX = math.avg(box.get_bottom(cBX), box.get_top(cBX))

            if math.sign(close[bl + 1] - mBX) != math.sign(low[bl] - mBX) or
math.sign(close[bl + 1] - mBX) != math.sign(high[bl] - mBX)
                cBX.delete()
                cLIQ.bx.remove(no)
            else
                cBX.set_right(b.i[bl])

if dpcS
    l = 0
    for pLev = pLta to pHta by pSpa
        if b.h[bl] >= pLev and b.l[bl] < pLev + pSpa
            aVPa.vs.set(l, aVPa.vs.get(l) + nzV[bl] * ((b.h[bl] - b.l[bl]) == 0 ? 1 :
pSpa / (b.h[bl] - b.l[bl])))
            l += 1

    if bl == ppLen

```



```

        ersten := math.avg(b.h[ppLen], b.l[ppLen])/pLta +
(aVPa.vs.indexof(aVPa.vs.max()) + .50) * pSpa
    else
        dPCa.push(line.new(b.i[bl] - 1, ersten, b.i[bl], pLta +
(aVPa.vs.indexof(aVPa.vs.max()) + .50) * pSpa, color = dpcC, width = 2))
        ersten := pLta + (aVPa.vs.indexof(aVPa.vs.max()) + .50) * pSpa

    if vpB
        aVP.vp.push(box.new(b.i[ppLen] - vpLen, pHst, b.i[ppLen], pLst, vpBC,
border_style = line.style_dotted, bgcolor = vpBC))

    if pcShw
        aPOC.push(box.new(b.i[ppLen] - vpLen, pLst + (pcL + .40) * pStp, b.i[ppLen],
pLst + (pcL + .60) * pStp, pcC, bgcolor = pcC ))

    vah = line.new(b.i[ppLen] - vpLen, pLst + (laP + 1.00) * pStp, b.i[ppLen], pLst +
(laP + 1.00) * pStp, xloc.bar_index, extend.none, vhShw ? vaHC : #00000000,
line.style_solid, 2)
    val = line.new(b.i[ppLen] - vpLen, pLst + (lbP + 0.00) * pStp, b.i[ppLen], pLst +
(lbP + 0.00) * pStp, xloc.bar_index, extend.none, vlShw ? vaLC : #00000000,
line.style_solid, 2)

    if vaB
        linefill.new(vah, val, vaBC)

    statTip = '\n -Traded Volume : ' + str.toString(tV, format.volume) + ' (' +
str.toString(vpLen - 1) + ' bars)' +
        '\n *Average Volume/Bar : ' + str.toString(tV / (vpLen - 1),
format.volume) +
        '\n\nProfile High : ' + str.toString(pHst, format.mintick) + ' ↑ %' +
str.toString((pHst - pLst) / pLst * 100, '#.##') +
        '\nProfile Low : ' + str.toString(pLst, format.mintick) + ' ↓ %' +
str.toString((pHst - pLst) / pHst * 100, '#.##') +
        '\n -Point Of Control : ' + str.toString(pLst + (pcL + .50) * pStp,
format.mintick) +
        '\n\nValue Area High : ' + str.toString(pLst + (laP + 1.00) * pStp,
format.mintick) +
        '\nValue Area Low : ' + str.toString(pLst + (lbP + 0.00) * pStp,
format.mintick) +
        '\n -Value Area Width : %' + str.toString(((pLst + (laP + 1.00) * pStp) -
(pLst + (lbP + 0.00) * pStp)) / (pHst - pLst) * 100, '#.##') +
        '\n\nNumber of Bars (Profile) : ' + str.toString(vpLen)

    if ppLev != 'Swing High/Low'
        uPI = ppLev == 'Value Area High/Low' ? pLst + (laP + 1.00) * pStp : pHst
        lPI = ppLev == 'Value Area High/Low' ? pLst + (lbP + 0.00) * pStp : pLst

        uTx = (ppP ? str.toString(uPI, format.mintick) : '') + (not na(pp_h) ? (ppC ?
(ppP ? ' ↑ %' : ' ↑ %') + str.toString((pp.h - pp.l) * 100 / pp.l, '#.##') : '') + (ppV ? (ppP
or ppC ? '\n' : '') + str.toString(tV, format.volume) : '') : '')

```

```

ITx = (ppP ? str.toString(IPI, format.mintick) : "") + (not na(pp_l) ? (ppC ? (ppP ? '
↓ %' : '↓ %') + str.toString((pp.h - pp.l) * 100 / pp.h, '#.##') : "") + (ppV ? (ppP or ppC ?
'\n' : "") + str.toString(tV, format.volume) : "") : "")

```

```

label.new(b.i[ppLen] - vpLen / 2, uPI, uTx, xloc.bar_index, yloc.price,
#00000000, label.style_label_down, chart.fg_color, ppS, text.align_center, ' Profile
High : ' + str.toString(pHst, format.mintick) + '\n %' + str.toString((pHst - pLst) / pLst *
100, '#.##') + ' higher than the Profile Low' + statTip)

```

```

label.new(b.i[ppLen] - vpLen / 2, IPI, ITx, xloc.bar_index, yloc.price, #00000000,
label.style_label_up , chart.fg_color, ppS, text.align_center, ' Profile Low : ' +
str.toString(pLst, format.mintick) + '\n %' + str.toString((pHst - pLst) / pHst * 100,
'#.##') + ' lower than the Profile High' + statTip)

```

```

else

```

```

    if not na(pp_h)

```

```

        label.new(b.i[ppLen], pp.h, (ppP ? str.toString(pp.h, format.mintick) : "") +
(ppC ? (ppP ? '↑ %' : '↑ %') + str.toString((pp.h - pp.l) * 100 / pp.l, '#.##') : "") + (ppV ?
(ppP or ppC ? '\n' : "") + str.toString(tV, format.volume) : ""), xloc.bar_index, yloc.price,
(not ppP and not ppC and not ppV ? chart.fg_color : #00000000),
label.style_label_down, chart.fg_color, (not ppP and not ppC and not ppV ?
size.tiny : ppS), text.align_center, 'Swing High : ' + str.toString(pp.h, format.mintick) +
'\n -Price Change : %' + str.toString((pp.h - pp.l) * 100 / pp.l, '#.##') + statTip)

```

```

        if not na(pp_l)

```

```

            label.new(b.i[ppLen], pp.l, (ppP ? str.toString(pp.l, format.mintick) : "") + (ppC ?
(ppP ? '↓ %' : '↓ %') + str.toString((pp.h - pp.l) * 100 / pp.h, '#.##') : "") + (ppV ? (ppP
or ppC ? '\n' : "") + str.toString(tV, format.volume) : ""), xloc.bar_index, yloc.price, (not
ppP and not ppC and not ppV ? chart.fg_color : #00000000), label.style_label_up ,
chart.fg_color, (not ppP and not ppC and not ppV ? size.tiny : ppS), text.align_center,
'Swing Low : ' + str.toString(pp.l, format.mintick) + '\n -Price Change : %' +
str.toString((pp.h - pp.l) * 100 / pp.h, '#.##') + statTip)

```

```

if pcShw and pcE != 'None'

```

```

    f_checkBreaches(aPOC, pcE)

```

```

for i = 0 to aLIQ.size() - 1

```

```

    x = aLIQ.get(i)

```

```

    int qBX = x.bx.size()

```

```

    for no = (qBX > 0 ? qBX - 1 : na) to 0

```

```

        if no < x.bx.size()

```

```

            cBX = x.bx.get(no)

```

```

            mBX = math.avg(box.get_bottom(cBX), box.get_top(cBX))

```

```

            if math.sign(close[1] - mBX) != math.sign(low - mBX) or math.sign(close[1] -
mBX) != math.sign(high - mBX)

```

```

                cBX.delete()

```

```

                x.bx.remove(no)

```

```

            else

```

```

                cBX.set_right(b.i)

```

```

vpLen := barstate.islast ? last_bar_index - pp.x + ppLen : 1
pHst := ta.highest(b.h, vpLen > 0 ? vpLen + 1 : 1)
pLst := ta.lowest(b.l, vpLen > 0 ? vpLen + 1 : 1)
pStp := (pHst - pLst) / vpLen
[_, _, tVd] = f_calcHL(vpLen, true, 0)

```

```

if barstate.islast and nzV and vpLen > 0 and pStp > 0

```

```

    if dVP.vp.size() > 0
        for i = 0 to dVP.vp.size() - 1
            dVP.vp.shift().delete()

```

```

    if dPOC.size() > 0
        for i = 0 to dPOC.size() - 1
            dPOC.shift().delete()

```

```

    tLIQ = dLIQ.shift()
    if tLIQ.bx.size() > 0
        for i = 0 to tLIQ.bx.size() - 1
            tLIQ.bx.shift().delete()
        tLIQ.b.shift()

```

```

    for bl = vpLen to 1 //1 to vpLen
        l = 0

```

```

        for pLev = pLst to pHst by pStp
            if b.h[bl] >= pLev and b.l[bl] < pLev + pStp
                aVP.vs.set(l, aVP.vs.get(l) + nzV[bl] * ((b.h[bl] - b.l[bl]) == 0 ? 1 : pStp /
(b.h[bl] - b.l[bl])))
                l += 1

```

```

        if dpcS
            if bl == last_bar_index - pp.x
                if dPCa.size() > 0
                    dPOC.push(line.new(b.i[bl], dPCa.get(dPCa.size() - 1).get_y2(), b.i[bl] +
1, pLst + (aVP.vs.indexOf(aVP.vs.max()) + .50) * pStp, color = dpcC, width = 2))
                else if bl < last_bar_index - pp.x
                    if dPOC.size() > 0
                        dPOC.push(line.new(b.i[bl], dPOC.get(dPOC.size() - 1).get_y2(), b.i[bl]
+ 1, pLst + (aVP.vs.indexOf(aVP.vs.max()) + .50) * pStp, color = dpcC, width = 2))

```

```

        dpcL := aVP.vs.indexOf(aVP.vs.max())
        ttV = aVP.vs.sum() * isVA
        va = aVP.vs.get(dpcL)
        laP := dpcL
        lbP := dpcL

```

```

        while va < ttV
            if lbP == 0 and laP == vpLev - 1

```

```

        break

    vaP = 0.
    if laP < vpLev - 1
        vaP := aVP.vs.get(laP + 1)

    vbP = 0.
    if lbP > 0
        vbP := aVP.vs.get(lbP - 1)

    if vbP == 0 and vaP == 0
        break

    if vaP >= vbP
        va += vaP
        laP += 1
    else
        vb += vbP
        lbP -= 1

    dLIQ.unshift(liquidity.new(array.new <bool> (na), array.new <box> (na)))
    cLIQ = dLIQ.get(0)

    for l = 0 to vpLev - 1
        if vpShw
            sbl = vpPlc == 'Right' ? b.i - int(aVP.vs.get(l) / aVP.vs.max() * vpLen *
vpWth) : b.i - vpLen
            ebl = vpPlc == 'Right' ? b.i : sbl + int( aVP.vs.get(l) / aVP.vs.max() * vpLen *
vpWth)
            dVP.vp.push(box.new(sbl, pLst + (l + 0.1) * pStp, ebl, pLst + (l + 0.9) * pStp, l
>= lbP and l <= laP ? vpTVC : vpVVC, bgcolor = l >= lbP and l <= laP ? vpTVC :
vpVVC))
            if liqUF
                if aVP.vs.get(l) / aVP.vs.max() < liqT
                    cLIQ.b.unshift(true)
                    cLIQ.bx.unshift(box.new(b.i, pLst + (l + 0.00) * pStp, b.i, pLst + (l + 1.00) *
pStp, border_color = color(na), bgcolor = liqC))
                else
                    cLIQ.bx.unshift(box.new(na, na, na, na))
                    cLIQ.b.unshift(false)

    for bl = 0 to vpLen
        int qBX = cLIQ.bx.size()

        for no = 0 to (qBX > 0 ? qBX - 1 : na)
            if no < cLIQ.bx.size()
                if cLIQ.b.get(no)
                    cBX = cLIQ.bx.get(no)
                    mBX = math.avg(cBX.get_bottom(), cBX.get_top())

```

```

        if math.sign(close[bl + 1] - mBX) != math.sign(low[bl] - mBX) or
math.sign(close[bl + 1] - mBX) != math.sign(high[bl] - mBX) or math.sign(close[bl +
1] - mBX) != math.sign(close[bl] - mBX)
            cBX.set_left(b.i[bl])
            cLIQ.b.set(no, false)

```

```

if vpB
    dVP.vp.push(box.new(b.i - vpLen, pHst, b.i, pLst, vpBC, bgcolor = vpBC))

```

```

if pcShw and not dpcS
    dVP.vp.push(box.new(b.i - vpLen, pLst + (dpcL + .40) * pStp, b.i, pLst + (dpcL +
.60) * pStp, pcC, bgcolor = pcC))

```

```

    vah = f_drawLineX(b.i - vpLen, pLst + (laP + 1.00) * pStp, b.i, pLst + (laP + 1.00) *
pStp, xloc.bar_index, extend.none, vhShw ? vaHC : #00000000, line.style_solid, 2)
    val = f_drawLineX(b.i - vpLen, pLst + (lbP + 0.00) * pStp, b.i, pLst + (lbP + 0.00) *
pStp, xloc.bar_index, extend.none, vlShw ? vaLC : #00000000, line.style_solid, 2)

```

```

if vaB
    linefill.new(vah, val, vaBC)

```

```

if ppLev != 'Swing High/Low'
    statTip = '\n -Traded Volume : ' + str.toString(tVd, format.volume) + ' (' +
str.toString(vpLen - 1) + ' bars)' +
    '\n *Average Volume/Bar : ' + str.toString(tVd / (vpLen - 1),
format.volume) +
    '\n\nProfile High : ' + str.toString(pHst, format.mintick) + ' ↑ %' +
str.toString((pHst - pLst) / pLst * 100, '#.##') +
    '\nProfile Low : ' + str.toString(pLst, format.mintick) + ' ↓ %' +
str.toString((pHst - pLst) / pHst * 100, '#.##') +
    '\n -Point Of Control : ' + str.toString(pLst + (dpcL + 0.50) * pStp,
format.mintick) +
    '\n\nValue Area High : ' + str.toString(pLst + (laP + 1.00) * pStp,
format.mintick) +
    '\nValue Area Low : ' + str.toString(pLst + (lbP + 0.00) * pStp,
format.mintick) +
    '\n -Value Area Width : %' + str.toString(((pLst + (laP + 1.00) * pStp) -
(pLst + (lbP + 0.00) * pStp)) / (pHst - pLst) * 100, '#.##') +
    '\n\nNumber of Bars (Profile) : ' + str.toString(vpLen) +
    (ppC ? '\n\n*price change caculated based on last swing high/low and
last price' : '')

```

```

uPI = ppLev == 'Value Area High/Low' ? pLst + (laP + 1.00) * pStp : pHst
IPI = ppLev == 'Value Area High/Low' ? pLst + (lbP + 0.00) * pStp : pLst

```

```

uTx = (ppP ? str.toString(uPI, format.mintick) : '') + (pp.s == 'L' ? (ppC ? (ppP ? '
↑ %' : '↑ %') + str.toString((close - pp.l) * 100 / pp.l, '#.##') + '*' : '') + (ppV ? (ppP or
ppC ? '\n' : '') + str.toString(tVd, format.volume) : '') : '')

```

```

ITx = (ppP ? str.toString(IPI, format.mintick) : '') + (pp.s == 'H' ? (ppC ? (ppP ? '
↓ %' : '↓ %') + str.toString((pp.h - close) * 100 / pp.h, '#.##') + '*' : '') + (ppV ? (ppP or

```

```
ppC ? '\n' : '') + str.tostring(tVd, format.volume) : '') : '')
```

```
    f_drawLabelX(b.i - vpLen / 2, uPI, uTx, xloc.bar_index, yloc.price, #00000000,  
label.style_label_down, chart.fg_color, ppS, text.align_center, 'Profile High : ' +  
str.tostring(pHst, format.mintick) + '\n %' + str.tostring((pHst - pLst) / pLst * 100,  
'#.#') + ' higher than the Profile Low' + statTip)
```

```
    f_drawLabelX(b.i - vpLen / 2, lPI, lTx, xloc.bar_index, yloc.price, #00000000,  
label.style_label_up, chart.fg_color, ppS, text.align_center, 'Profile Low : ' +  
str.tostring(pLst, format.mintick) + '\n %' + str.tostring((pHst - pLst) / pHst * 100,  
'#.#') + ' lower than the Profile High' + statTip)
```

```
//-----}
```

```
//Alerts
```

```
//-----{
```

```
priceTxt = str.tostring(close, format.mintick)
```

```
tickerTxt = syminfo.ticker
```

```
if ta.cross(close, pLst + (dpcL + 0.50) * pStp) and pcShw
```

```
    alert(tickerTxt + ' : Swings Volume Profile : Price touches/crosses Point Of Control  
Line, price ' + priceTxt)
```

```
if ta.cross(close, pLst + (laP + 1.00) * pStp) and vhShw
```

```
    alert(tickerTxt + ' : Swings Volume Profile : Price touches/crosses Value Area High  
Line, price ' + priceTxt)
```

```
if ta.cross(close, pLst + (lbP + 0.00) * pStp) and vlShw
```

```
    alert(tickerTxt + ' : Swings Volume Profile : Price touches/crosses Value Area Low  
Line, price ' + priceTxt)
```

```
//-----}
```